

Adversarial test set & agent improvement plan

Why this exists

The shipped agent scores **0.859** on the 200-session public set. That number is optimistic. The public and private sets are sampled *uniformly* from the frozen catalog, and the shipped pipeline's dominant signal — verbatim constraint-span coverage in the reranker (`src/rerank.py`) — only works when the constraints the simulated customer discloses form a near-unique fingerprint of the target.

For a large share of the catalog they don't:

Catalog property	Share of 50k catalog
"new" override intent is a bare material / short phrase, target in a big cluster	17.3%
has a price but ≤ 1 distinctive non-material constraint	13.5%
every soft preference is Amazon boilerplate	11.6%
shares (category, material, colour) with ≥ 40 other products	7.1%
own cluster small but (material, colour) pair catalog-wide common	5.5%
features + details collapse to one phrase or the bare title	0.5%

Having the property is not the same as failing — most such targets still carry a usable secondary span. But a uniform sample dilutes these cases; this set **concentrates** them so the weak stages are forced to carry the session.

Everything here is inside Track 4 scope (`docs/competition_specification.md`): real `parent_asin` targets from the frozen read-only catalog, the published session schema, and the unmodified evaluator. The hidden intent card is still built by the evaluator's own `intent_card()`; the generator only chooses *which* targets and *which* scenario.

Files

File	What it is
<code>tools/hard_cases.py</code>	Deterministic generator. Scans the catalog, buckets every product by adversarial property, samples N per bucket, writes sessions. --run also scores the shipped agent and re-groups the per-session results by bucket.
<code>data/hard_set.jsonl</code>	96 sessions (16 × 6 buckets), public-set schema.

```
python3 tools/hard_cases.py --run # generate + score, per-bucket table
python3 -m evaluator.local_evaluator --dataset data/hard_set.jsonl # official scorer
```

Result — shipped agent

bucket	n	hit@10	mrr	mttc	score
homogeneous_cluster	16	0.625	0.493	5.88	0.563
budget_only_signal	16	0.750	0.656	4.88	0.694
boilerplate_soft	16	0.875	0.875	3.88	0.843
degenerate_card	16	0.562	0.427	6.19	0.505
generic_override	16	0.688	0.591	5.81	0.625
cross_category_collision	16	0.938	0.833	3.19	0.875

ALL	96	0.740	0.646	4.97	0.684
public-set reference		0.940	0.791	3.41	0.859

For comparison, the public set's own `difficulty_bucket = "hard"` sessions still score 0.867 hit@10 — the organizer's difficulty label does **not** capture these failure modes. Four of the six buckets are materially harder than anything in the public set.

Failure mechanism (one dominant cause)

Traced through `retrieve()` → `rerank()` for the missed sessions, the picture is almost always the same. When the disclosed constraints reduce to something like `['cotton', '100% Cotton', 'Imported', 'Button closure']`:

1. **Retrieval** (`src/retrieval.py`) places the target at rank **25–250** in the

pool of 300. A bag-of-words OR query over ~4 non-discriminative tokens cannot separate it from every other cotton button-down shirt.

2. **Reranking** (`src/rerank.py`) does **nothing**, because span coverage is

identical across the hundreds of candidates that also literally contain "100 cotton", "imported" and "button closure". Observed rerank position $\approx$ retrieval position (e.g. 237 $\rightarrow$ 237, 246 $\rightarrow$ 246). The reranker is a no-op whenever every candidate covers the same generic spans.

3. The target's **category is known** ("Shirts Casual Button-Down Shirts") and its

facets are extracted (`src/facets.py` builds material/colour/brand/price/ category for every product) — but **no retrieval route and no ranking signal reads either one**. FacetStore is constructed at `starter/agent.py:75` and, under the default FixedPolicy, never queried at runtime.

`degenerate_card` is bimodal: when the one disclosed phrase happens to be the full title ("AOQ Men cargo pants Army Camo ... Khaki, 46") the target is rank 1; when it is "Imported" or "color: blue" the reranker has no span at all and the target floats wherever BM25 left it.

`boilerplate_soft` and `cross_category_collision` barely move — the agent is genuinely robust there, because those targets still tend to carry one distinctive primary constraint.

Two secondary findings:

- **The override down-weight is inert.** `apply_override()` sets every prior

`Utterance.weight` to `PRE_OVERRIDE_WEIGHT = 0.35`, but that value is only ever read as a boolean (`focused_text()` keeps utterances with `weight >= 1.0`). `full_text()` — which drives the main terms route — ignores weight entirely, and `query_spans()` feeds every non-turn-1 utterance to the reranker regardless of weight. So 0.35 behaves identically to any value in (0, 1); the "sweep the override weight" idea in IMPLEMENTATION.pdf §S3 cannot pay off as the code stands.

- **Budget constraints are triply dead.** For the 13.5% of targets with a price,

the disclosed "budget around \$x" (a) barely helps FTS retrieval (just the digits), (b) can never substring-match in `rerank()` because price is not in the product text blob, and (c) FixedPolicy never asks budget anyway.

Improvement plan (priority order)

1. Category-anchored retrieval route + ranking boost — attacks cluster / override / degenerate

The opening message always names a coarse category. Add it as a real signal, two places:

- **Retrieval route:** a third RRF route in `retrieve()` restricted to the

category subtree (FTS query AND-ed with the category tokens, or a post-filter on `index.products[asin]["categories"]`). Fused at RRF weight ~0.5. This alone should pull rank-250 targets into the top 50 where the reranker can act.

- **Rerank signal:** in `rerank()`, add `+ w_cat * (candidate category leaf == opening category leaf)`.

`src/facets.py:_category_leaf()` already computes the leaf.

Expected: the largest single gain. Directly targets the three worst buckets and the traced belt/shirt/boot failures. Est. +0.05–0.10 on the stress set, neutral on the public set (category already agrees there).

2. Facet-agreement ranking signal — attacks cluster / cross-category

FacetStore is already built and unused. In `rerank()` add `+ w_facet * agreement(candidate_facets, disclosed_facets)` where disclosed facets come from running `src/facets.extract`-style matching over `state.query_spans()`. Material/colour agreement breaks ties inside a cluster that span coverage cannot. Add a **negative** term too: a candidate whose material is explicitly *different* from a stated one is pushed down.

Expected: +0.02–0.05 on the stress set, targets MRR (30% of score). Low risk — it is additive over code that already exists.

3. Make the pool deeper or the rerank depth = pool size — cheap safety net

Retrieval pool is 300, `RerankConfig.depth` is 200. Targets fused into positions 201–300 are never reranked (they sit in the untouched tail). Either set `depth = pool_size`, or widen the pool to 500 for the first two turns. One-line change; removes a hard cliff for the cluster/override buckets.

4. Confidence-aware stopping on a stagnating session — attacks all misses

The agent shows the same list every turn from turn 3 once evidence stops arriving. On a session heading for a miss it should notice (`state.last_turn_productive == False` for 2+ turns, shortlist unchanged) and switch strategy — diversify the list across sub-categories/brands, or ask a *disambiguating* question generated from where the top candidates disagree (`InfoGainPolicy` already computes facet distributions over the pool). Given a miss→hit is worth ~2.6× a rank improvement, trading rank for coverage on low-confidence sessions is likely net-positive.

5. Fix or delete the override weight — correctness

Either make `full_text()` / `query_spans()` actually honour `Utterance.weight` (repeat or scale down-weighted tokens), or drop `PRE_OVERRIDE_WEIGHT` and rely on the `focused_text()` route alone. Then the §S3 sweep becomes meaningful. The `generic_override` bucket (0.625) is where this shows up.

6. Learn the rerank weights — once 1–2 add new features

After adding category + facet signals, `rerank()` has ~5 weights set by hand. One offline logistic-regression pass over the ~176 known-correct public sessions (standard library, no network) fits them jointly. IMPLEMENTATION.pdf §10 already flags this as the highest-EV cross-cutting change; it should come *after* the new features exist, not before.

Regenerating / tuning the set

`tools/hard_cases.py --per-bucket N --seed S`. The bucket definitions (`classify_product`) and thresholds (`cluster ≥ 40`, `matcol ≥ 700`, price present, etc.) are all near the top of the file. To add a bucket: add a tag in `classify_product`, a `BUCKET_SCENARIO` entry, and it flows through generation and scoring automatically.